Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Dersin tarihi 13 Nisan 2021 Salı. Ders Zoom üzerinden, İngilizce yapıldı.
  - Asıl anlatım yaklaşık 00:04:30'da başlar. Kayıtta iki ara var (yaklaşık 00:49-01:02 ve 02:02-02:13).
- Bağlamdan bağımsız dillere giriş (context-free languages, CFL)
  - $a^n b^n$ ve ww^R düzenli değildir ama bağlamdan bağımsızdır.
  - CFL'ler bağlamdan bağımsız gramerlerle (context-free grammar, CFG) ya da yığıtlı otomatlarla (pushdown automata, PDA) tanımlanır. Önce gramerler işlendi.
- İngilizce gramer benzetmesiyle gramer kavramı
- Değişkenler (variables), uç simgeler (terminals), kurallar (rules / productions), başlangıç değişkeni
- Türetme (derivation) ve gramerin dili
- Kısa gösterimler: $\Rightarrow^*$ ve $|$
- CFG'nin biçimsel tanımı (dörtlü)
- Bağlamdan bağımsız dilin tanımı
- Gramerden dil bulma örnekleri: $a^n b^n$, çift uzunluklu palindromlar, eşleşen parantezler
- En soldan ve en sağdan türetme (leftmost / rightmost derivation)
- Türetme ağacı (derivation tree / parse tree)
- Belirsizlik (ambiguity): aritmetik ifade grameri, if-else (sarkan else) sorunu
- Belirsizliğin giderilmesi ve doğası gereği belirsiz diller (inherently ambiguous languages)
- Quiz ve vize duyurusu

Hocanın Özellikle Vurguladığı Kısımlar

- Her türetme başlangıç değişkeniyle başlar.
  - Başlangıç değişkeni genellikle en üstte ve en solda yazılır, çoğunlukla S'dir.
- Kuralda tek ok ($\rightarrow$), türetme adımında kalın ok ($\Rightarrow$) kullanılır.
- Bir gramerin dilini görmek için önce birkaç dizgi türetin. Sonra deseni küme gösterimiyle yazın.
- Belirsiz gramer derleyicide sorun çıkarır.
  - Bir kodun tek bir anlamı olmalıdır. Ayrıştırma ağacı kodun anlamıdır.
- Belirsizlik tanımında dikkat:
  - Ağaçlarla bakıyorsanız: iki farklı türetme ağacı olması yeterlidir, en soldan olmaları gerekmez.
  - Türetmelerle bakıyorsanız: iki farklı **en soldan** türetme gerekir.
- En soldan ve en sağdan türetmenin aynı ağacı vermesi belirsizliğin olmadığını göstermez. Hoca bu yanlışı ders içinde düzeltti.
- Haftaya ders saatinde quiz var. Bir önceki gibi test olacak.
- Vize 8. haftada.

Kısa Tekrar Notları

- CFG dörtlüsü: $G = (V, T, S, P)$.
  - V: değişkenler kümesi, T: uç simgeler kümesi, S: başlangıç değişkeni, P: kurallar kümesi.
  - Sipser aynı tanımı (V, Σ, R, S) olarak yazar.

- Büyük harfler değişken, küçük harfler uç simgedir.
- Bir değişken uç simgelere, değişkenlere ya da ikisinin karışık bir dizisine dönüşebilir.
- $u \Rightarrow^* v$: u'dan v'ye sıfır ya da daha fazla adımda ulaşılır.
- $S \to aSb \mid \varepsilon$, iki ayrı kuralın kısa yazımıdır.
- Gramerin dili: $L(G) = \{w \in T^* \mid S \Rightarrow^* w\}$.
- L bağlamdan bağımsızdır $\iff$ $L(G) = L$ olan bir CFG G vardır.
- En soldan türetme: her adımda en soldaki değişken değiştirilir. En sağdan türetme: en sağdaki.
- Türetme ağacının yaprakları soldan sağa okununca türetilen dizgi (yield) çıkar. ε yaprakları boş dizgidir.
- Belirsiz gramer: dilde iki farklı türetme ağacı (ya da iki farklı en soldan türetmesi) olan bir dizgi vardır.
- Doğası gereği belirsiz dil: bu dili üreten **her** gramer belirsizdir.

Detaylı Açıklamalar

İngilizce gramer benzetmesi

Küçük bir gramer:

- `<sentence>` → `<noun-phrase>` `<predicate>`
- `<noun-phrase>` → `<article>` `<noun>`
- `<predicate>` → `<verb>`
- `<article>` → a | the
- `<noun>` → cat | dog
- `<verb>` → runs | sleeps

Türetme: `<sentence>` ⇒ `<noun-phrase>` `<predicate>` ⇒ `<article>` `<noun>` `<verb>` ⇒ ⋯ ⇒ "the dog runs".

- Üç değişkenin her birinin iki seçeneği var. Bu yüzden dilde $2 \cdot 2 \cdot 2 = 8$ cümle vardır.
- "runs", "dog" gibi simgelerden başka bir şey türetilemez. Bunlar uç simgelerdir.

Örnek: $S \to aSb \mid \varepsilon$

- ab: $S \Rightarrow aSb \Rightarrow ab$.
- $aabb$: $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$.
- S kendi kuralında yeniden geçtiği için (bir döngü) sonsuz sayıda dizgi türetilir.
- $L(G) = \{a^n b^n \mid n \geq 0\}$. $n = 0$ için ε elde edilir.
- Biçimsel tanım: $V = \{S\}$, $T = \{a, b\}$, başlangıç S, $P = \{S \to aSb, S \to \varepsilon\}$.
- Bu yüzden $a^n b^n$ bağlamdan bağımsız bir dildir.

Örnek: $S \to aSa \mid bSb \mid \varepsilon$

- $S \Rightarrow aSa \Rightarrow abSba \Rightarrow abbSbba \Rightarrow abbbba$.
- Soldaki kısmın tersi sağda yazılır.
- $L(G) = \{ww^R \mid w \in \{a, b\}^*\}$: çift uzunluklu palindromlar.

Örnek: $S \to aSb \mid SS \mid \varepsilon$ **(eşleşen parantezler)**

- a yerine "(", b yerine ")" yazılınca dizgiler doğru eşleşmiş parantezlere dönüşür.
- $L(G) = \{w \mid n_a(w) = n_b(w)$ ve w'nin her v öneki (prefix) için $n_a(v) \geq n_b(v)\}$.
  - $n_a(w)$: w'deki a sayısı.
  - Yalnızca eşit sayı koşulu yetmez. ba gibi dizgileri dışlamak için önek koşulu gerekir.
- Önek: dizginin başından başlayan bir alt dizgi. Dizginin kendisi de bir önekidir.

En soldan ve en sağdan türetme

Gramer (kareden): $S \to AB$, $A \to aaA \mid \varepsilon$, $B \to Bb \mid \varepsilon$. Bu gramerin dili $(aa)^* b^*$'dır.

- En soldan türetme (aab): $S \Rightarrow AB \Rightarrow aaAB \Rightarrow aaB \Rightarrow aaBb \Rightarrow aab$.

- En sağdan türetme (aab): $S \Rightarrow AB \Rightarrow ABb \Rightarrow Ab \Rightarrow aaAb \Rightarrow aab$.
- İki türetme dizisi farklıdır ama aynı türetme ağacını verir.
- En soldan ya da en sağdan demek, birden çok seçenek olduğunda sırayı belirli hâle getirir.

Türetme ağacı

- Her türetme adımında değiştirilen değişkenin altına, kuralın sağ tarafındaki simgeler dal olarak eklenir.
- Ağacın yaprakları soldan sağa okunur. Örnekte yapraklar $a, a, \varepsilon, \varepsilon, b$; dizgi aab.
- Derleyiciler programı ayrıştırırken (parse) bu ağaçları kurar.

Belirsizlik örneği: aritmetik ifadeler

Gramer: $E \to E + E \mid E \times E \mid (E) \mid a$.

$a + a \times a$ için iki farklı en soldan türetme ağacı vardır:

1. Önce $E \to E + E$: $a + (a \times a)$.
2. Önce $E \to E \times E$: $(a + a) \times a$.

- $a = 2$ alınırsa birinci ağaç $2 + 2 \cdot 2 = 6$, ikinci ağaç $(2 + 2) \cdot 2 = 8$ verir.
- Gramerde işlem önceliği (precedence) yoktur. Derleyici hangi kuralı önce seçerse sonuç ona göre değişir. Bu gramer belirsizdir.

Belirsizliği giderme: öncelikli gramer

Aynı dili üreten belirsiz olmayan gramer:

- $E \to E + T \mid T$
- $T \to T \times F \mid F$
- $F \to (E) \mid a$
- $a + a \times a$ için tek ağaç vardır: $E \Rightarrow E + T$, $E \Rightarrow T \Rightarrow F \Rightarrow a$, $T \Rightarrow T \times F$, ...
- $E \Rightarrow T$ ile başlanırsa $+$ işaretini üretmek için parantez gerekir. Dizgide parantez olmadığı için bu yol tıkanır.
- Çarpma, toplamadan daha derinde üretildiği için önce yapılır.

Belirsizlik örneği: sarkan else (dangling else)

Gramer: `<stmt>` → if `<expr>` then `<stmt>` | if `<expr>` then `<stmt>` else `<stmt>`. Uç simgeler: if, then, else.

- Hoca ders içinde bir eksiği düzeltti: if ifadesi de bir `<stmt>` olabilmeli. Aksi hâlde iç içe if türetilemez.
- Dizgi: if E1 then if E2 then S1 else S2.
  1. Ağaç 1: else içteki if'e bağlanır: if E1 then (if E2 then S1 else S2).
  2. Ağaç 2: else dıştaki if'e bağlanır: if E1 then (if E2 then S1) else S2.
- Çözüm dile göre değişir:
  - Perl gibi diller süslü parantezi zorunlu tutar. Böylece else'in hangi if'e ait olduğu bellidir.
  - C süslü parantezsiz yazmaya izin verir. Programcı dikkatli olmalıdır.

Doğası gereği belirsiz diller

Bazen belirsizlik giderilemez. Bu dillere doğası gereği belirsiz denir.

Örnek (kareden): $L_1 = \{a^n b^n c^m\} \cup \{a^n b^m c^m\}$, $n, m \geq 0$.

- L_1'de b sayısı ya a sayısına ya da c sayısına eşittir. Örnekler: c, ab, a, bc, $aabbc$, $abbcc$.
- L_1 için bir gramer:
  - $S \to S_1 \mid S_2$

  - $S_1 \to S_1 c \mid A$
  - $A \to aAb \mid \varepsilon$
  - $S_2 \to aS_2 \mid B$
  - $B \to bBc \mid \varepsilon$
- $a^n b^n c^n$ biçimindeki dizgiler hem S_1 hem S_2 yolundan türetilir. Bu yüzden bu gramer belirsizdir. Hoca, L_1 için yazılabilecek her gramerin belirsiz olduğunu söyledi ve konuyu evde çalışmaya bıraktı.

Duyurular

- Quiz 1: 5 soru, her biri 20 puan, yanlış doğruyu götürmüyor. Notlar ders sonunda ders duvarında paylaşıldı.
- Quiz 2 haftaya ders saatinde, test şeklinde.
- Vize 8. haftada (dönem 8 Mart'ta başladı).
- Çalışma kâğıdı 2 (geçen yılın 2. ödevi) ders duvarına yüklendi.
  - Büyük kısmı PDA ve CFL pompalama önsavı gerektirdiği için şu an çözülemez. Vizeden önce işe yarayacak.
  - Şimdilik kitaptan çalışın. Bir dilin bağlamdan bağımsız olduğunu göstermek için gramerini yazmanız gerekir.

Ders sonrası notlar

- Hoca $L_2 = \{a^n b^n c^n \mid n \geq 0\}$ için "belirsiz, gramer bulunamaz" dedi. Doğrusu: L_2 bağlamdan bağımsız **değildir**, onu üreten hiçbir CFG yoktur. "Doğası gereği belirsiz" kavramı yalnızca bağlamdan bağımsız diller için kullanılır. L_2 bu yüzden belirsiz dil örneği değildir. CFL'ler için pompalama önsavıyla ispatlanır (Sipser Örnek 2.36).
- L_1'in doğası gereği belirsiz olduğu doğrudur. Sipser bu dili Bölüm 2'de doğası gereği belirsiz dil örneği olarak verir.
- $\Rightarrow^*$ için hoca önce "bir ya da daha fazla adım" dedi, sonra "sıfır ya da daha fazla" diye düzeltti. Doğrusu sıfır ya da daha fazladır.
- Eşleşen parantez dilinde hoca önek koşulunu önce "büyük" diye söyledi. Doğru koşul "büyük ya da eşit"tir ($n_a(v) \geq n_b(v)$).
- Belirsiz olmayan gramer için önemli olan, her dizginin tek bir türetme ağacı (tek bir en soldan türetmesi) olmasıdır. En soldan ve en sağdan türetmelerin aynı ağacı vermesi her gramerde olur; belirsizliği göstermez.
- Altyazıda "F can be E or A" diye geçen kural $F \to (E) \mid a$'dır.
- C'de sarkan else, en yakın eşleşmemiş if'e bağlanır. Belirsizlik bu dil kuralıyla çözülür.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz. Ders İngilizce anlatılmıştır; altyazı da büyük ölçüde İngilizcedir.